BUBBLE-BLUE: private group messaging over an open Bluetooth LE mesh

Paul-Maxence Baraton[1], Nadjib Achir[1], Philippe Jacquet[1], Wilfred Doré[2]
[1]Inria, France [2]Qualcomm, France paul-maxence.baraton@inria.fr

arXiv:2509.00967

1 The idea

Where infrastructure is absent, saturated or not trusted (a crowd, a disaster site, an event), devices can still reach each other over Bluetooth. Existing Bluetooth meshes either require provisioning, so a stranger cannot relay, or let every relay read every message [2–4]. BUBBLE-BLUE separates two things:

- the **mesh** is open: any board may join and relay, including traffic it cannot decrypt, and nobody pairs;
- a **bubble** is a group defined by a key: only its members read its messages.

2 How a message travels

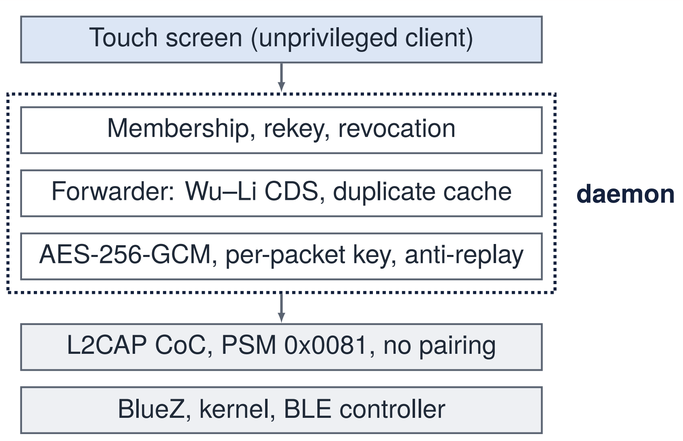


Figure 1. One daemon per board owns the radio; screens are clients.

- **Links.** A board advertises an anonymous identifier, a relay flag and its L2CAP address; neighbours open a channel at security level *low*: no pairing, no bond.
- **Relays.** Each board applies the Wu–Li rule [7] to its two-hop view; the marked boards form a connected dominating set, the only ones to rebroadcast.

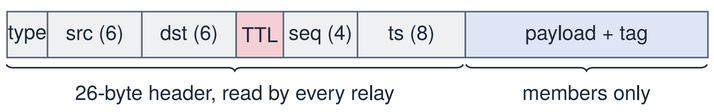


Figure 2. Frame format; sizes in bytes. Duplicates are dropped on (source, sequence).

3 Security

What each party learns.

	radio listener	relay, not a member	member
addresses, hops left, time	yes	yes	yes
size and timing of traffic	yes	yes	yes
text of a group message	no	no	**yes**
forge or alter a message	no	no	yes

Table 1. Payload privacy, not metadata privacy: the Bluetooth address is stable and the header is in clear.

Group messages. With K_e the group key of epoch e, every frame gets its own key, so a zero GCM nonce is never reused:

$$K_{\text{pkt}} = \text{HKDF}(K_e;\ src \,\|\, seq \,\|\, ts)$$
$$c = \text{GCM}_{K_{\text{pkt}}}(m;\ header)$$

The header is authenticated as associated data, except TTL, which relays decrement.

At the stand

Three Thundercomm RUBIK Pi 3 screens (Qualcomm QCS6490) and three Arduino UNO Q relays (Qualcomm QRB2210). Try it:

- send from a screen: the message crosses a relay outside the group;
- exclude a board: it keeps relaying, and stops reading;
- restart the mesh: watch links and group come back.

Keys between members

Each board A holds an X25519 secret a and an Ed25519 signing key. Every 30 s it floods a signed ANNOUNCE binding its address to $A = aG$. Two members then share a key without ever having met:

$$K_{AB}^{(e)} = \text{HKDF}(aB,\ e) = \text{HKDF}(bA,\ e)$$

since $aB = bA = abG$; nobody else can compute it.

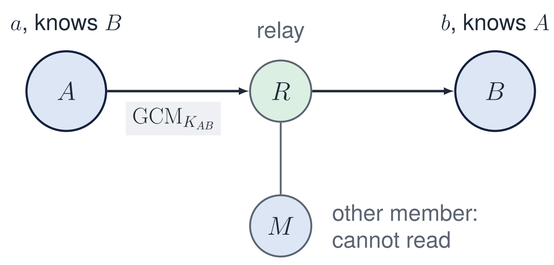


Figure 3. A direct message from A to B crosses the mesh like any frame, under a per-packet key derived from K_{AB}: relays and the other members forward it but cannot read it.

Removing a member V. The coordinator C (the member with the smallest anonymous identifier) draws a new group key K_{e+1} and a one-time key k, and floods one frame. Each remaining member i gets k wrapped under the pairwise key it shares with C:

$$w_i = \text{HKDF}(K_{Ci}^{(e)};\ src \,\|\, seq \,\|\, ts)$$
$$case_i = tag_i \,\|\, \text{GCM}_{w_i}(k) \qquad (i \neq V)$$
$$body = \text{GCM}_k\big(e{+}1 \,\|\, K_{e+1} \,\|\, V \,\|\, \sigma_C\big)$$

where σ_C is C's Ed25519 signature over the epoch, the key and the revoked key. There is no case for V.

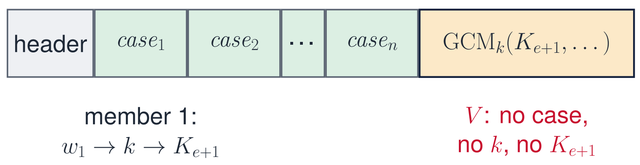


Figure 4. The rekey frame. Relays, V included, forward it unchanged; only listed members can open it. The number of cases is visible on the air.

Each member checks the signature, that the signer is the coordinator, and that the epoch increases, then switches to K_{e+1}. V keeps relaying the new traffic but cannot read it. A newcomer can be excluded only once its keys have been announced: wait one ANNOUNCE period after a join.

What we accept in this prototype.

Who	What it can still do
Non-member relay	drop or delay frames, rewrite TTL.
Active outsider	flood junk; pre-empt a (source, sequence) pair in relay caches.
Rogue member	grind its identifier to become coordinator.
Observer of a rekey	count the remaining members.

Table 2. Availability attacks remain; the confidentiality and authenticity of the payload do not depend on any relay.

Checked. On the boards, an excluded screen did not display the next message (2 trials of 2). In simulation, from 6 to 30 nodes, no remaining member was stranded and nothing leaked to the excluded one.

4 Four hops across a building

The stand fits on a table. Spread through the building, about 15 m apart with one relay off, the boards formed a line by themselves.

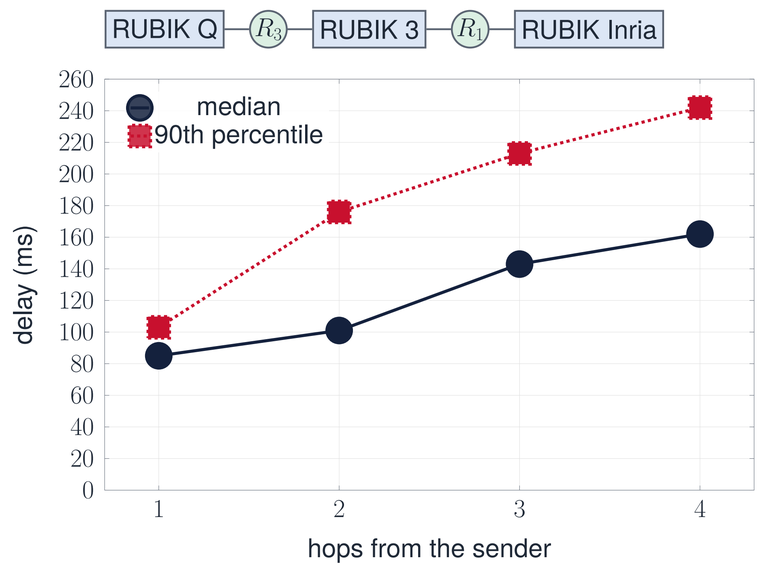


Figure 5. Send-to-display delay, on-board clocks within 50 ms. The far end received 76 of 79 messages, missing only those sent while the line formed. Carried away, its link held to about 30 m.

5 8.2 hours on battery

The stand on battery, one message every 30 s: 988 messages, 4 915 expected receptions.

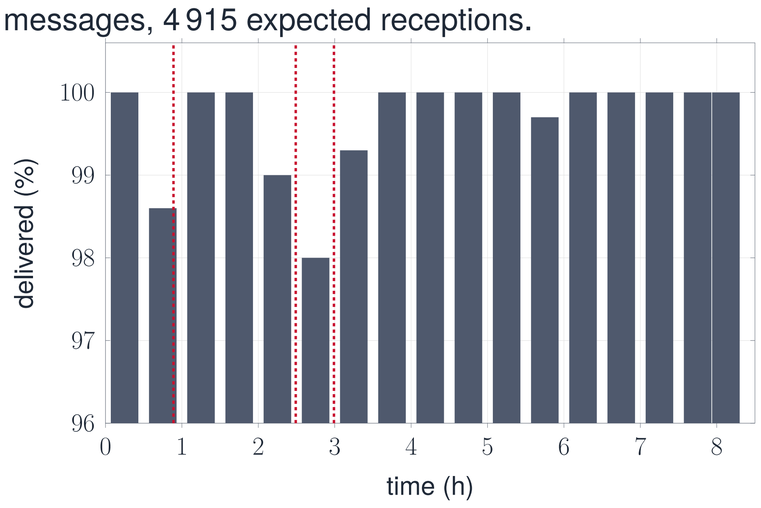


Figure 6. Delivery per 30 minutes. Dashed: a relay's Bluetooth controller stopped answering and was reloaded without a reboot. Screens received 100 %, relays 99.3 %; median delay 80 ms, 99th percentile 188 ms; 6 of 6 boards still up when stopped.

6 Limits and next steps

- A restarted sender restarts its sequence numbers, which the receivers' anti-replay windows reject: they must survive a restart.
- Two earlier defects, also found by measuring, are fixed with regression tests: ANNOUNCE and data shared one anti-replay window, and a sparse mesh could elect several coordinators.
- TTL 10 spans about twenty nodes in simulation; on a line, one relay is a single point of failure.
- Next: an Android port, and replaying the protocol on the contact graph the link logs already record, to see how election and revocation behave when people move.

[1] N. Achir, P. Jacquet, BUBBLE-BLUE, arXiv:2509.00967, 2025. [2] Bluetooth SIG, Mesh Profile 1.0, 2017. [3] J. Dorsey et al., BitChat, 2025. [4] M. R. Albrecht et al., Breaking Bridgefy, CT-RSA 2021. [7] J. Wu, H. Li, connected dominating sets, DIAL-M 1999. Boards provided by the Qualcomm University Platforms program.